

PORTADA

Arquitectura de Software

Semana 7 — Catálogo POSA: Procesamiento por Lotes y Cliente/Servidor

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

Muchos bancos grandes del mundo todavía cierran el día... a las 3 a.m.



El ciclo batch nocturno del núcleo bancario (core bancario)

Patrón	Procesamiento por lotes (Batch Processing)
Dónde se usa	Sistemas núcleo (<i>core banking</i>) de bancos grandes, muchos todavía sobre mainframe
Tecnología típica	Programas COBOL orquestados por scripts JCL (Job Control Language) sobre sistemas operativos de mainframe (ej. z/OS de IBM)
Cuándo corre	De madrugada, en una ventana batch de baja actividad de usuarios (típicamente entre la medianoche y las primeras horas de la mañana)
Qué procesa	Todas las transacciones acumuladas durante el día: pagos, transferencias, consignaciones, retiros

Modelo ampliamente documentado en la industria financiera: sigue vigente en muchas instituciones grandes junto a sus canales digitales en tiempo real.

CASO · QUÉ PASA ESA NOCHE

Un pipeline de jobs, uno detrás de otro

Durante el día, el banco registra las transacciones en un sistema transaccional en tiempo real, pero **no las cierra ni las concilia de inmediato**. Esa tarea se deja para el ciclo batch de la noche, que ejecuta una secuencia de **jobs** (programas por lotes) donde la salida de uno alimenta al siguiente:

1. **Consolidación:** reunir todas las transacciones del día en un solo lote.
2. **Conciliación de cuentas:** verificar que débitos y créditos cuadren entre sistemas.
3. **Cálculo de intereses:** aplicar tasas sobre saldos de cuentas de ahorro, préstamos y tarjetas.
4. **Actualización de saldos:** dejar el saldo "oficial" de cada cuenta listo para el día siguiente.
5. **Generación de extractos:** producir los reportes y estados de cuenta del período.

CASO · EL PROBLEMA QUE RESUELVE

Por qué procesar así, y no transacción por transacción

El **volumen** de transacciones diarias de un banco grande es masivo, y varias operaciones —como el cálculo de intereses o la conciliación— solo tienen sentido sobre el **conjunto completo** del día, no sobre cada movimiento aislado. Procesar por lotes, en una ventana de baja actividad, permite:

- Usar toda la capacidad del mainframe sin competir con las transacciones en vivo de los usuarios.
- Garantizar **consistencia**: todo el banco cierra el día con los mismos números.
- Recuperarse de errores relanzando un job puntual, sin rehacer todo el día transacción por transacción.

CASO · LA LIMITACIÓN QUE ARRASTRA

El precio: nada está "cerrado" hasta la madrugada

Mientras el ciclo batch no corre, los saldos "oficiales" de intereses, comisiones y conciliaciones **no reflejan el día en curso** — por eso a veces una transferencia aparece "pendiente" varias horas. Esta latencia es aceptable para procesos contables, pero cada vez más difícil de sostener cuando:

- La operación es **global 24/7** y ya no hay una franja horaria de "baja actividad" clara.
- Los usuarios esperan confirmaciones y saldos **en tiempo real**, no al día siguiente.

Esta tensión —lotes vs. tiempo real— es exactamente lo que empuja a la industria hacia arquitecturas de streaming/event-driven, que veremos más adelante en el curso.

TRANSICIÓN

El banco no inventó el batch: aplicó un patrón general

El ciclo nocturno bancario es un ejemplo concreto de un patrón de procesamiento mucho más general, presente también en facturación masiva, nómina, reportes de fin de mes y cargas de datos entre sistemas.

Formalicemos qué es el procesamiento por lotes como patrón, y cuándo conviene usarlo.

CONCEPTO

Procesamiento por lotes (Batch Processing)

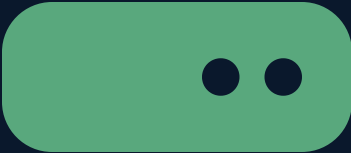
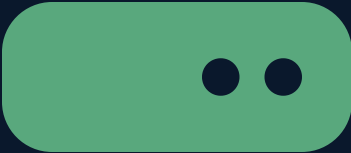
Es el patrón que consiste en **agrupar grandes volúmenes de datos en bloques ("lotes")** y procesarlos completos, **sin interacción del usuario en tiempo real**, normalmente en una ventana de tiempo programada.

- **Job:** unidad de trabajo por lotes (ej. "calcular intereses del día").
- **Ventana batch:** franja horaria reservada para ejecutar el ciclo completo de jobs.
- **Pipeline secuencial:** los jobs se encadenan — la salida de uno es la entrada del siguiente, como se vio en el caso bancario.

CONCEPTO

Ventajas y limitaciones del batch

Ventajas	Limitaciones
Eficiente para volúmenes muy grandes de datos	No es tiempo real: introduce latencia hasta el próximo ciclo
Fácil de razonar: secuencia clara de pasos, uno tras otro	La ventana batch es cada vez más difícil de sostener con operación 24/7 global
Aprovecha capacidad de cómputo en horas de baja demanda	Un fallo a mitad del ciclo puede retrasar todo el resto de jobs encadenados
Simplifica auditoría: se procesa y se cierra un conjunto completo y trazable	Escala peor frente a la presión moderna de datos "siempre frescos"



TRANSICIÓN

Del "cuándo" procesar al "dónde" se ejecuta cada parte

El batch resuelve un problema de **tiempo** (cuándo procesar). El segundo patrón de hoy resuelve un problema distinto: **dónde**, físicamente, se ejecuta cada responsabilidad de una aplicación — en la máquina del usuario, en un servidor, o repartida entre varios servidores.

Ese es el patrón arquitectónico Cliente/Servidor, y su evolución histórica en 2, 3 y N capas.

CONCEPTO

Cliente/Servidor: el patrón base

Divide una aplicación en dos roles que se comunican por red: un **cliente**, que solicita servicios, y un **servidor**, que los provee (datos, cómputo, lógica de negocio). El diseño concreto de este patrón cambió mucho a lo largo de la historia según **cuántas capas** —niveles físicamente separables— se usaran para repartir presentación, lógica de negocio y datos.

A esta evolución en capas se le conoce como arquitectura de **2, 3 y N capas**.

CONCEPTO · EVOLUCIÓN 1

2 capas: el "cliente gordo" (fat client)

El cliente concentra **presentación y lógica de negocio** juntas, y se conecta directamente al servidor de base de datos — sin nada intermedio.

- **Capa 1:** aplicación de escritorio (interfaz + reglas de negocio).
- **Capa 2:** servidor de base de datos.

Caso real: aplicaciones empresariales de escritorio de los años 90, construidas con herramientas como **PowerBuilder** o **Visual Basic**, conectadas directo a un servidor **Sybase** u **Oracle** — muy comunes en banca y ERPs de esa época.

CONCEPTO · EVOLUCIÓN 1 (LÍMITE)

El problema del cliente gordo

- **Actualizar la lógica de negocio** exige reinstalar el ejecutable en cada máquina cliente — costoso a gran escala.
- Cada cliente abre su propia conexión a la base de datos: **difícil de escalar** a miles de usuarios simultáneos.
- La lógica de negocio queda esparcida en cada instalación, no en un solo lugar controlado.

Estas limitaciones motivaron separar la lógica de negocio del cliente hacia un nivel intermedio propio: nace el modelo de 3 capas.

CONCEPTO · EVOLUCIÓN 2

3 capas: presentación, negocio y datos separados

Se separan explícitamente tres niveles, cada uno con su responsabilidad y, típicamente, en su propia máquina física:

- **Capa de presentación:** cliente ligero (navegador), sin lógica de negocio.
- **Capa de lógica de negocio:** un **servidor de aplicaciones** centralizado que procesa las reglas del sistema.
- **Capa de datos:** servidor de base de datos.

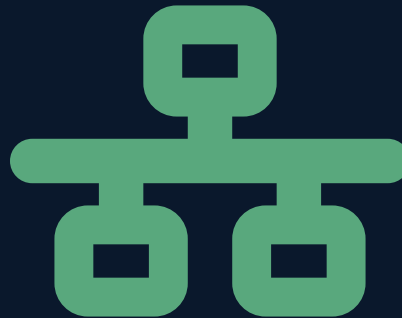
Caso real: el modelo que popularizó la primera ola de aplicaciones web empresariales de fines de los 90 — navegador → servidor de aplicaciones (Java EE o ASP) → base de datos.

N capas: generalizando el modelo

N capas extiende el modelo de 3 capas agregando **más niveles intermedios** según lo necesite el sistema, en vez de un único servidor de aplicaciones monolítico:

- Una capa de **servicios** separada de la capa de presentación.
- Una capa de **integración** con sistemas externos.
- Una capa de **caché distribuido** entre la aplicación y la base de datos.

Llevado al extremo, este mismo impulso de separar responsabilidades en más niveles independientes es lo que conecta con los microservicios (Semana 8) y con los estilos arquitectónicos que veremos en la Semana 9.



CONCEPTO

2, 3 y N capas, comparadas

Modelo	Dónde vive la lógica de negocio	Ejemplo real
2 capas	En el cliente ("cliente gordo")	Apps de escritorio en PowerBuilder/VB + Sybase u Oracle (años 90)
3 capas	En un servidor de aplicaciones central	Navegador + Java EE/ASP + base de datos (primeras apps web empresariales)
N capas	Repartida en varios servicios especializados	Capas de servicios, integración y caché; llevado al extremo, microservicios

CONTRA EJEMPLO

Cliente/Servidor en capas no es lo mismo que MVC

Es fácil confundirlos porque ambos hablan de "separar responsabilidades", pero responden preguntas distintas:

- **Cliente/Servidor en capas** es sobre **dónde** físicamente se ejecuta cada responsabilidad — distribución física y de red entre máquinas (cliente, servidor de aplicaciones, servidor de datos).
- **MVC** (visto la sesión anterior) es sobre **cómo** se organiza el código *dentro* de una sola capa — separar Modelo, Vista y Controlador dentro del mismo proceso o servicio.

Son ortogonales, no lo mismo: un servidor de aplicaciones de 3 capas perfectamente puede internamente estar organizado con MVC — las dos decisiones se toman por separado.

INTERACCIÓN

Ubiquen el patrón

Para cada escenario, digan si es más cercano a 2, 3 o N capas, y por qué:

1. Una hoja de cálculo empresarial de los años 90 que se conecta directo a un servidor Oracle desde cada PC.
2. Una app web moderna con frontend en React, un backend con capa de servicios, una capa de caché con Redis y una base de datos separada.
3. Un sistema con navegador, un único servidor de aplicaciones Java EE, y una base de datos — sin nada más entre medio.

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. El **procesamiento por lotes** agrupa grandes volúmenes de datos y los procesa sin interacción en tiempo real, en una ventana programada — el ciclo batch nocturno bancario en mainframe (COBOL/JCL) es el caso real más extendido.
2. Su ventaja es la eficiencia sobre grandes volúmenes; su límite es la **latencia** frente a la presión moderna de operar 24/7 en tiempo real.
3. **Cliente/Servidor** resuelve un problema distinto: dónde físicamente se ejecuta cada responsabilidad de la aplicación.
4. Su evolución histórica va de **2 capas** (cliente gordo) a **3 capas** (servidor de aplicaciones central) a **N capas** (servicios especializados, antesala de microservicios).
5. Cliente/Servidor en capas (distribución física) y **MVC** (organización del código dentro de una capa) son **ortogonales** — no son el mismo tipo de decisión.

CIERRE

Para la próxima sesión

- Repasar la diferencia entre distribución física (cliente/servidor en capas) y organización interna del código (MVC), porque la próxima sesión sigue explorando cómo se organiza un sistema por dentro.
- La próxima sesión del catálogo POSA cubre dos patrones más: **Layers (Capas)** y **Microkernel**.

